

Ex. No.: 5

Experiment Title: System Calls Programming

Aim:

To experiment with system calls using `fork()`, `execlp()`, and `pid()` functions in a C program.

Algorithm:

1. Start

- Include required header files: `stdio.h`, `stdlib.h`, and `unistd.h`.

2. Variable Declaration

- Declare an integer variable `pid` to hold the process ID.

3. Create a Process

- Call `fork()` to create a new process. Store return value in `pid`.
 - If `fork()` returns `-1`: Forking failed.
 - If `fork()` returns `0`: This is the child process.
 - If `fork()` returns a positive value: This is the parent process.

4. Print Statement Executed Twice

- Print: `THIS LINE EXECUTED TWICE` (this runs in both child and parent).

5. Check for Process Creation Failure

- If `pid == -1`:
 - Print `CHILD PROCESS NOT CREATED`

- Exit the program.

6. Child Process Execution

- If `pid == 0`:
 - Print Child Process ID: `getpid()`
 - Print Parent Process ID: `getppid()`

7. Parent Process Execution

- If `pid > 0`:
 - Print Parent Process ID: `getpid()`
 - Print Parent's Parent Process ID: `getppid()`

8. Final Print Statement

- Print: `IT CAN BE EXECUTED TWICE`

9. End

Program Code:

```
c
CopyEdit
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int pid;
    pid = fork();

    printf("THIS LINE EXECUTED TWICE\n");
```

```
if (pid == -1) {
    printf("CHILD PROCESS NOT CREATED\n");
    exit(0);
} else if (pid == 0) {
    printf("CHILD PROCESS\n");
    printf("Process ID: %d\n", getpid());
    printf("Parent Process ID: %d\n", getppid());
} else {
    printf("PARENT PROCESS\n");
    printf("Process ID: %d\n", getpid());
    printf("Parent's Parent Process ID: %d\n", getppid());
}

printf("IT CAN BE EXECUTED TWICE\n");

return 0;
}
```

Sample Output:

yaml

CopyEdit

THIS LINE EXECUTED TWICE

PARENT PROCESS

Process ID: 10234

Parent's Parent Process ID: 4567

IT CAN BE EXECUTED TWICE

THIS LINE EXECUTED TWICE

CHILD PROCESS

Process ID: 10235

Parent Process ID: 10234

IT CAN BE EXECUTED TWICE

Result:

The program was executed successfully. It demonstrated how system calls like `fork()`, `getpid()`, and `getppid()` work by creating a child process and displaying their respective process IDs.